

제 10 주:

연결체인 큐



강 지 훈

jhkang@cnu.ac.kr

충남대학교 컴퓨터공학과



[제 10 주]

환형 연결 체인으로 구현된 큐



실습 목표

■ 이론적 관점

- 큐의 개념

■ 구현적 관점

- 연결체인을 이용하여 환형 큐로 구현하는 방법

과제에서 해결할 문제



□ [프로그램 10]

- 키보드에서 문자를 반복 입력 받는다.
 - 한번에 한 개의 문자를 입력 받는다.

- 입력의 종료 조건
 - ! 키를 치면 더 이상 입력을 받지 않는다.

- 매번 한 개의 문자를 입력 받을 때 마다, 문자에 따라 정해진 일을 한다.

이 과제에서 필요한 객체는?

- ApplicationController
- AppView
- Model
 - Interface "Queue<T>"
 - Class "CircularlyLinkedListQueue<T> implements Queue<T>"
 - Class "Node<T>"
 - Enum "MessageID"

□ 문자에 따라 해야 할 일 [1]

- 영문자 ('A' ~ 'Z', 'a' ~ 'z'): 큐에 삽입한다.
 - 다음과 같은 메시지를 내보낸다. (입력된 영문자가 'x' 라면)
 - ◆ "[EnQueue] 삽입된 원소는 'x' 입니다."
 - 만일 큐가 full 이면 다음과 같은 메시지를 내보낸다.
 - ◆ ERROR : 큐가 꽉 차서 삽입이 불가능합니다.

- '!': 큐의 원소를 모두 삭제하고, 프로그램을 종료한다.
 - "큐 입력을 종료합니다" 메시지를 내보낸다.
 - 큐를 모두 비운다.
 - 또한 다음과 같은 통계 자료를 출력한다.
 - ◆ ".....입력된 문자는 모두 16 개입니다."
 - ◆ ".....정상 처리된 문자는 15 개입니다."
 - ◆ ".....무시된 문자는 1 개입니다."
 - ◆ ".....삽입된 문자는 6 개입니다."

□ 문자에 따라 해야 할 일 [2]

- 숫자문자('0' ~ '9'): 해당 수 만큼 큐에서 삭제한다.
 - 먼저 다음 메시지를 내보낸다. ('3'이 입력되었다면)
 - ◆ [RemoveN] 3 개의 원소를 삭제하려고 합니다.
 - 매번 삭제될 때마다 다음과 같은 메시지를 내보낸다. (삭제된 원소가 'x' 라면)
 - ◆ "[DeQueue] 삭제된 원소는 'x' 입니다.
 - 삭제를 하려는데 큐가 empty 이면, 다음과 같은 메시지를 내보내고 삭제를 멈춘다.
 - ◆ "ERROR : 큐에 원소가 없습니다."

- '-': 큐의 front 원소를 삭제한다.
 - 다음과 같은 메시지를 내보낸다. (front 원소가 'r' 라면)
 - ◆ "[DeQueue] 삭제된 원소는 'r' 입니다.
 - 삭제를 하려는데 큐가 empty 이면, 다음과 같은 메시지를 내보내고 삭제를 멈춘다.
 - ◆ "ERROR : 큐에 원소가 없습니다."

□ 문자에 따라 해야 할 일 [3]

- '#': 큐의 길이를 다음과 같이 출력한다. (현재 큐에 5 개의 원소가 있다면)
 - "[Size] 큐에는 현재 5 개의 원소가 있습니다."
- '/': 큐의 내용을 front 부터 rear 까지 다음과 같이 차례로 출력한다.
 - "[Queue] <Front> A b c d E f <Rear>"
- '^': 큐의 front 원소의 값을 출력한다. 큐는 변하지 않는다. (큐의 front 원소가 'f' 라면)
 - "[Front] 맨 앞 원소는 'f' 입니다."
 - 큐가 empty 이면 다음과 같은 메시지를 내보낸다.
 - ◆ "ERROR : 큐에 원소가 없습니다."

□ 문자에 따라 해야 할 일 [4]

- 그 밖의 문자들: 다음과 같이 출력하고 무시한다.
 - "ERROR : 의미 없는 문자가 입력되었습니다."

□ 출력의 예

```
> 프로그램을 시작합니다.
[큐 입력을 시작합니다.]
- 문자를 입력하시오: -
[Empty] 큐에 원소가 없습니다.
- 문자를 입력하시오: A
[EnQueue] 삽입된 원소는 A 입니다.
- 문자를 입력하시오: x
[EnQueue] 삽입된 원소는 x 입니다.
- 문자를 입력하시오: #
[Size] 큐에는 현재 2개의 원소가 있습니다.
- 문자를 입력하시오: h
[EnQueue] 삽입된 원소는 h 입니다.
- 문자를 입력하시오: /
[Queue] <Front> A x h <Rear>
- 문자를 입력하시오: W
[EnQueue] 삽입된 원소는 W 입니다.
- 문자를 입력하시오: Z
ERROR : 큐가 꽉 차서 삽입이 불가능합니다.
- 문자를 입력하시오: -
[DeQueue] 삭제된 원소는 A 입니다.
- 문자를 입력하시오: ^
[Front] 맨 앞 원소는 x입니다.
- 문자를 입력하시오: /
[Queue] <Front> x h W <Rear>
```

```
- 문자를 입력하시오: 2
[RemoveN] 2개의 원소를 삭제하려고 합니다.
[DeQueue] 삭제된 원소는 x 입니다.
[DeQueue] 삭제된 원소는 h 입니다.
- 문자를 입력하시오: 3
[RemoveN] 3개의 원소를 삭제하려고 합니다.
[DeQueue] 삭제된 원소는 W 입니다.
[Empty] 큐에 원소가 없습니다.
[Empty] 큐에 원소가 없습니다.
- 문자를 입력하시오: B
[EnQueue] 삽입된 원소는 B 입니다.
- 문자를 입력하시오: =
ERROR : 의미 없는 문자가 입력 되었습니다.
- 문자를 입력하시오: e
[EnQueue] 삽입된 원소는 e 입니다.
- 문자를 입력하시오: /
[Queue] <Front> B e <Rear>
- 문자를 입력하시오: !
[큐 입력을 종료합니다.]
[DeQueue] 삭제된 원소는 B 입니다.
[DeQueue] 삭제된 원소는 e 입니다.
....입력된 문자는 모두 17 개입니다.
....정상 처리된 문자는 16 개입니다.
....무시된 문자는 1 개입니다.
....삽입된 문자는 6 개입니다.
```

```
>프로그램을 종료합니다.
```

main()

□ main()을 위한 class

```
public class DS1_10_학번_이름 {  
    public static void main (String[] args)  
    {  
        ApplicationController appController = new ApplicationController() ;  
        // ApplicationController 가 실질적인 main class 이다.  
        appController.run() ;  
        // 여기 main()에서는 앱 실행이 시작되도록 해주는 일이 전부이다.  
    }  
}
```

Class "AppController"



□ Class "AppController" 의 구현 [1]

```

public class AppController {
    // 비공개 변수들
    private AppView _appView ;
    private CircularLinkedQueue<Character> _queue ; // 지난 주와 다른 부분
    private int _inputChars ; // 입력된 문자의 개수
    private int _ignoredChars ; // 무시된 문자의 개수
    private int _addedChars ; // 삽입된 문자의 개수

    // 생성자
    public AppController()
    {
        this._appView = new AppView() ;
        this.initCharCounts() ;
    }

    // 비공개함수의 구현
    .....

    // 공개함수의 구현
    .....
} // End of class "AppController"

```

□ ApplicationController: 비공개 함수

- [제 9 주] 의 인스턴스 변수 `_queue`의 선언에서 자료형으로 `class CircularArrayQueue` " 대신에 `class "CircularlyLinkedQueue"` 를 사용하는 것을 제외하고는, 모든 코드는 동일하다.
- 그러므로 [제 9 주] 의 `class "AppController"`를 복사하여 수정하여 사용한다.

Class "AppView"



□ Class "AppView" 의 공개함수

- [제 9 주] 실습 과제의 것을 복사하여 그대로 사용한다.

Enum "MessageID"

□ Enum "MessageID"

- [제 9 주] 실습 과제의 것을 그대로 복사하여 사용한다.

Class "Node<T>" 의 구현



□ Node: 인스턴스 변수

```
public class Node <T> {  
    private T        _element ;  
    private Node<T>  _next ;  
}
```

□ Node: 생성자, getter/setter

- 생성자: 각 생성자에 따라 달리 주어지는 인자를 사용하여 그에 맞게 `_element` 와 `_next` 를 초기화
 - `public Node ()`
 - `public Node (T givenElement)`
 - `public Node (T givenElement, Node<T> givenNode)`
- Getter / Setter for "`_element`"
 - `public T element()`
 - ◆ `this._element` 를 반환
 - `public void setElement (T newElement)`
 - ◆ `this._element` 에 `newElement` 를 저장
- Getter / Setter for "`_next`"
 - `public Node<T> next()`
 - ◆ `this._next` 를 반환
 - `public void setNext(Node<T> newNext)`
 - ◆ `this._next` 에 `newNext` 를 저장

Interface "Queue"



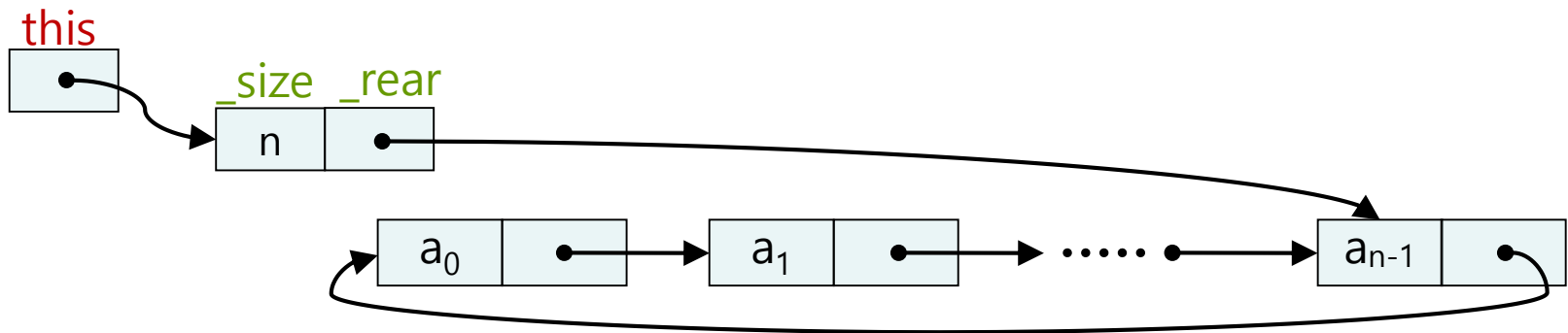
□ Interface "Queue"

```
public interface Queue<T>
{
    public boolean isEmpty ();
    public boolean isFull () ;
    public boolean enqueue (T anElement) ;
    public T dequeue () ;
    public T frontElement () ;
}
```

Class “CircularlyLinkedQueue”



□ 객체의 모습



□ CircularlyLinkedListQueue: 인스턴스 변수

```
public class CircularlyLinkedListQueue<T>
    implements Queue<T>
{
    private int        _size ;
    private Node<T>    _rear ;
```

□ CircularlyLinkedListQueue: 생성자, getter

■ 생성자

- public CircularlyLinkedListQueue ()
 - ◆ this._size 와 this._rear 의 초기화

■ public int capacity ()

- 큐가 수용할 수 있는 원소의 최대 개수를 돌려 주어야 한다. CircularlyLinkedListQueue 에서는 이 값을 어떻게 해야 할까?

■ public int size ()

- 현재 큐가 가지고 있는 원소의 개수, 즉 this._size 의 값을 돌려준다.

□ CircularlyLinkedQueue: 공개함수

■ 상태 알아보기

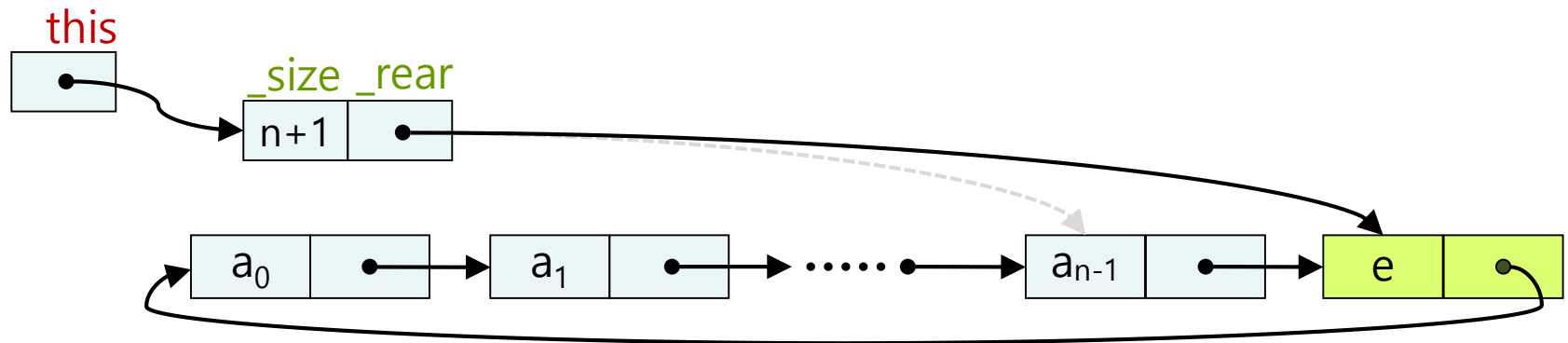
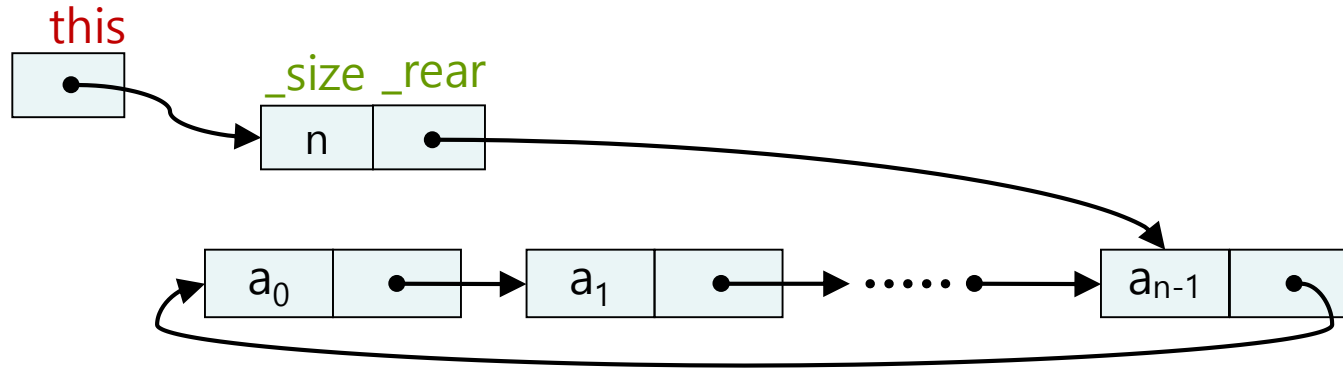
- public boolean isEmpty ()
 - ◆ this._rear 의 값이 null 이면 true 를 반환
- public boolean isFull ()
 - ◆ 연결 체인이 가득 차게 되는 경우는?
- public T frontElement ()
 - ◆ 큐가 비어 있으면 null 을 돌려준다.
 - ◆ 비어 있지 않으면, _rear 의 다음 node 가 front node 이다.
이 front node 의 element 를 돌려준다.

□ CircularlyLinkedListQueue: 삽입 함수 enqueue() 의 구현

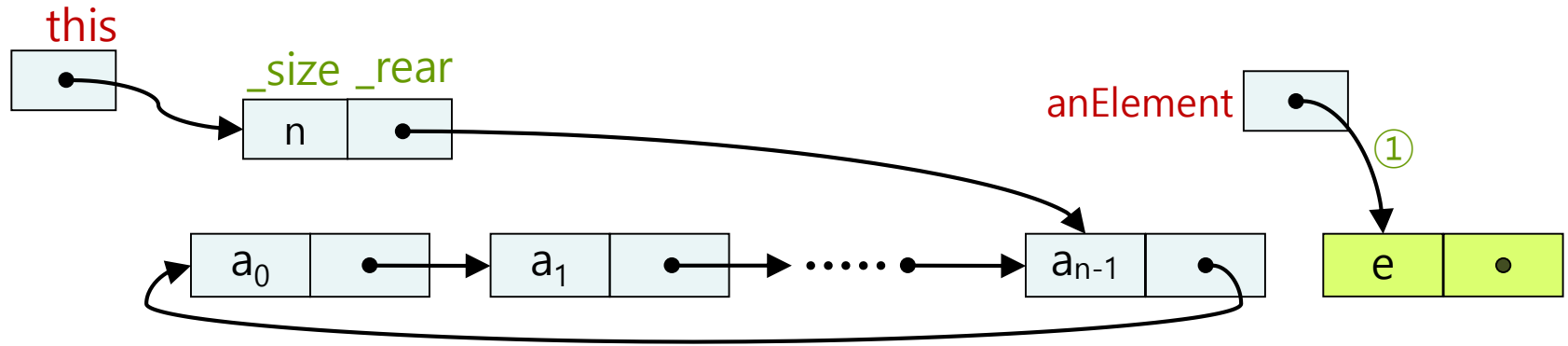
■ public boolean enqueue (T anElement)

- 가득 차 있을 경우:
 - ◆ false를 반환하여야 하나, 연결체인으로 구현되므로 이 경우는 발생하지 않는다.
- 가득 차 있지 않을 경우
 - ◆ empty 라면?
 - ◆ empty 가 아니라면?

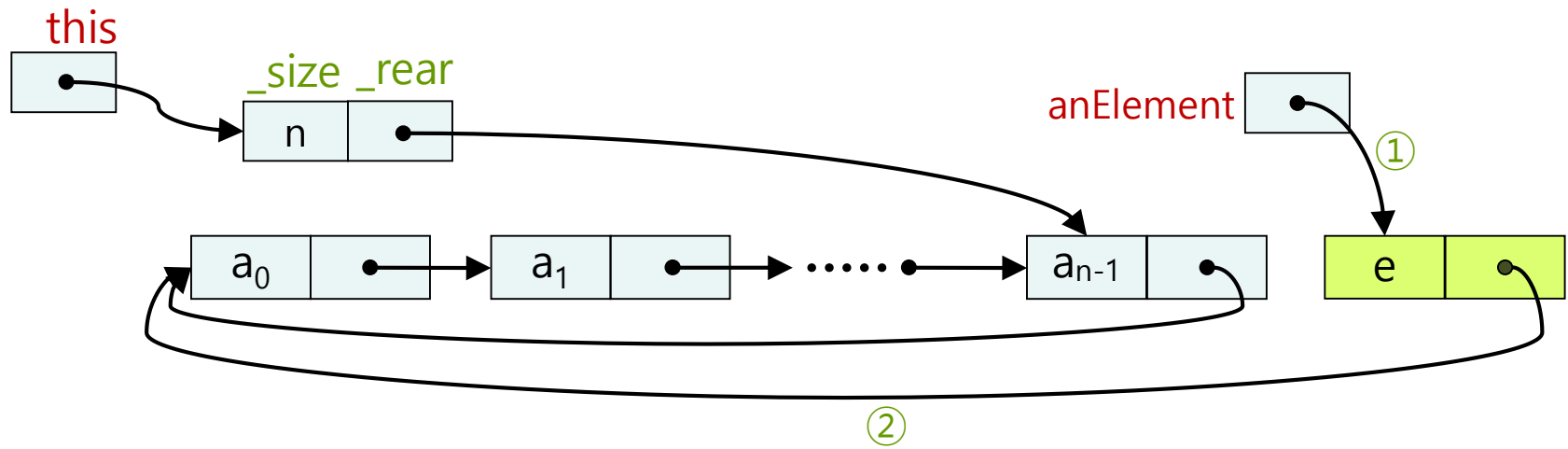
□ Rear에 삽입: 전과 후의 모습



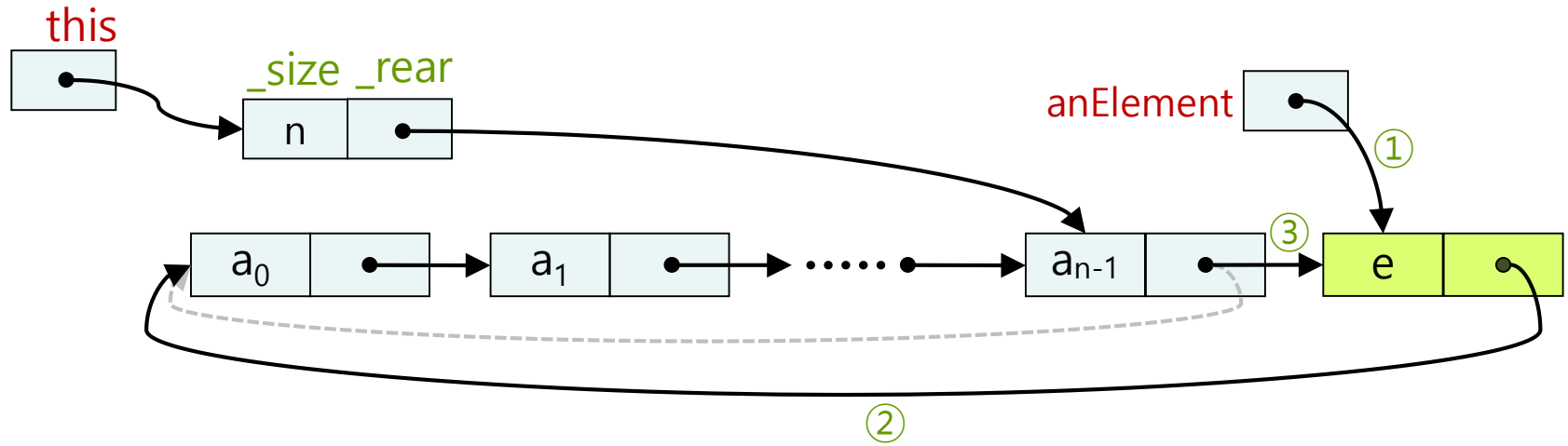
□ Rear에 삽입: 단계 1



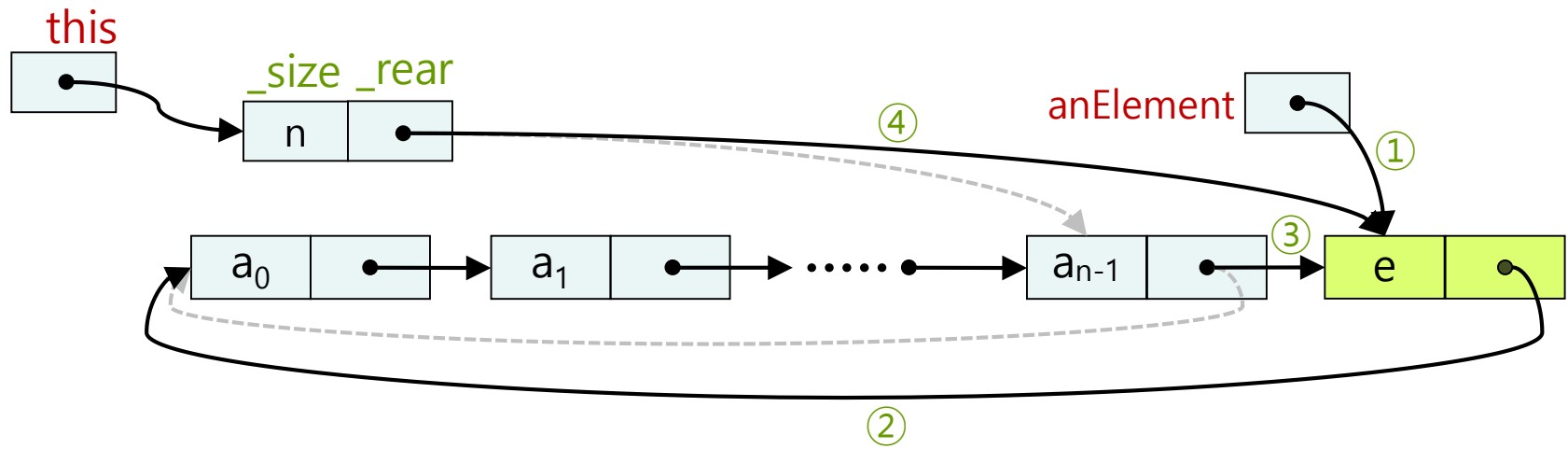
□ Rear에 삽입: 단계 2



□ Rear에 삽입: 단계 3

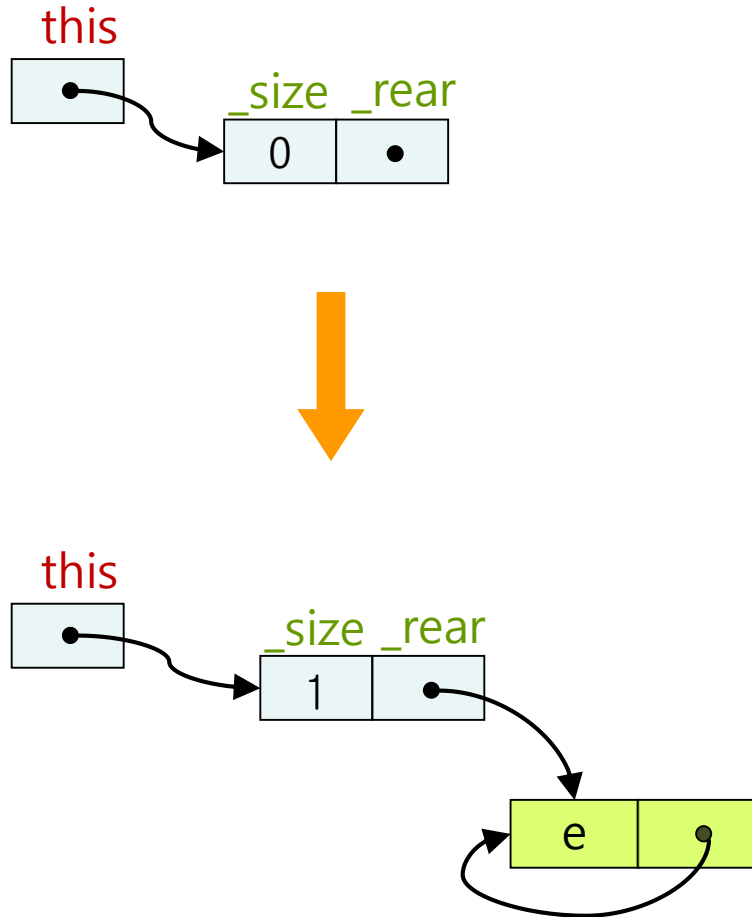


□ Rear에 삽입: 단계 4



□ 삽입: 리스트가 비어 있다면 ?

- 비어 있지 않은 경우와 동일한 코드로 가능할까?

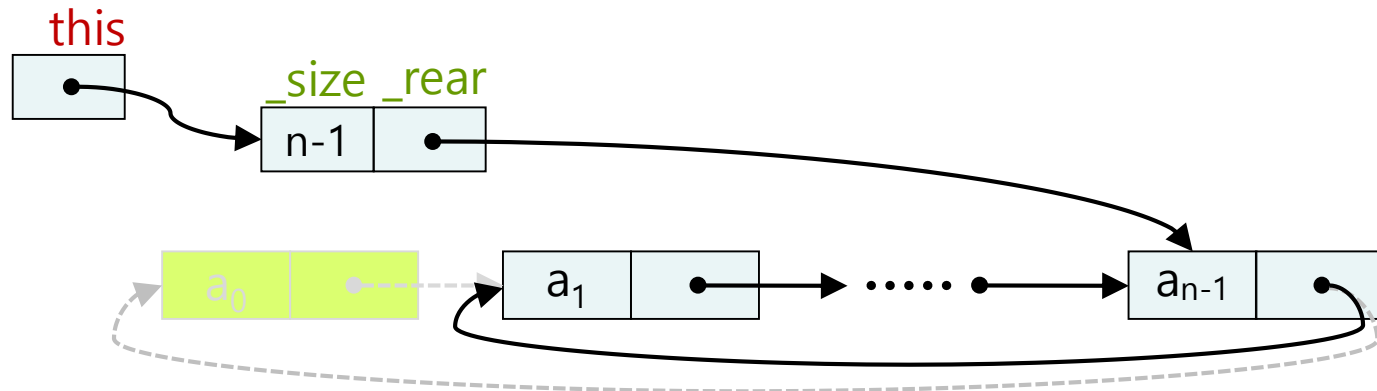
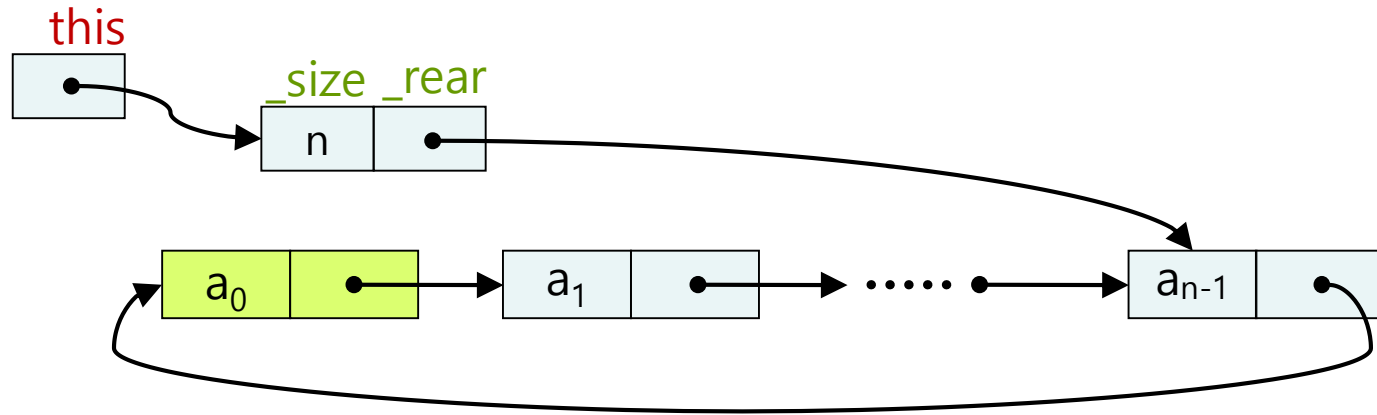


□ CircularlyLinkedListQueue: 삭제 함수 deQueue() 의 구현

■ public T deQueue ()

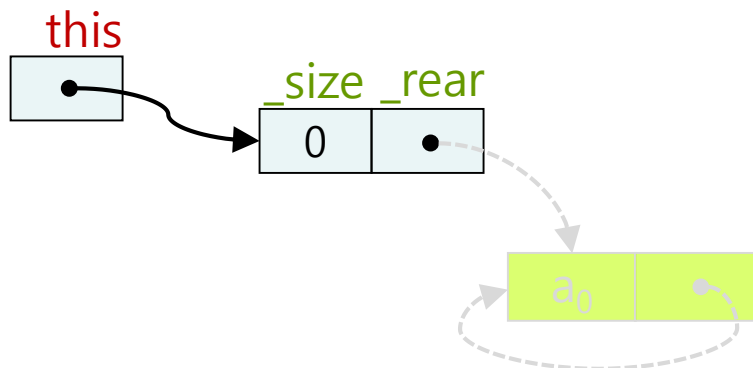
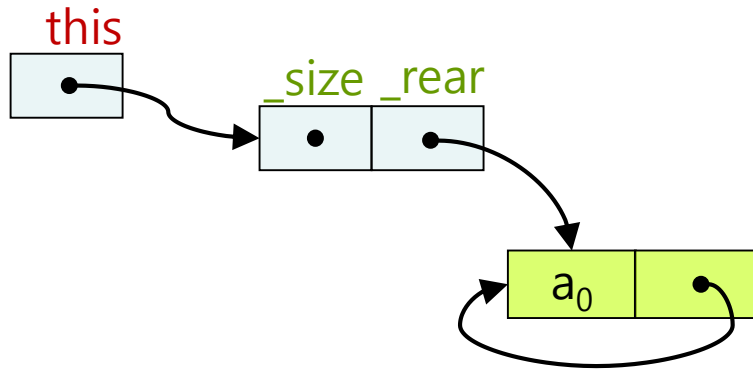
- ◆ 비어 있으면: null 을 돌려준다.
- ◆ 만약 비어 있지 않으면: front element 를 삭제하여 돌려준다.

□ Front 에서 삭제



❑ 삭제: 리스트에 노드가 한 개만 있다면?

- 노드가 두 개 이상인 경우에 삭제하는 코드를 그대로 사용할 수 있을까?



□ CircularlyLinkedListQueue:

■ public void clear ()

- 처음 생성했을 때의 empty queue 상태와 동일하게 만든다.

■ public T elementAt (int anOrder)

- front 노드의 위치 값이 0 이다.
- front 노드는 rear 노드의 다음 노드이다. 그러므로 front 노드는 다음과 같이 얻을 수 있다.
 - ◆ Node front = this._rear.next() ;
- front 노드부터 세어서 anOrder 위치의 노드의 원소를 돌려준다.
- anOrder 의 값은 0 부터 (this._size - 1) 까지 사이의 값이다.

요약

□ 생각할 점

■ 연결 체인

- 삽입과 삭제
- 리스트 구현에 있어서, 배열과의 차이점과 장단점

■ 배열로 구현한 큐와 연결체인으로 구현한 큐의 차이점은 무엇인가?

- 차이점을 분석하여 보고서에 쓰시오.

■ Queue 를 interface 로 선언하는 이유는?

- 큐를 다른 방식으로 구현할 경우를 생각해 보자.

보고서 작성과 제출



□ PDF 보고서 작성 방법

■ 겉장

- 제목: 자료구조 실습 보고서
- [제10주] 연결체인 큐
- 제출일
- 학번/이름

■ 내용

1. 프로그램 설명서
 1. 주요 알고리즘 /자료구조 /기타
 2. 함수 설명서
 3. 종합 설명서
2. 프로그램 장단점 분석
3. 실행 결과 분석
 1. 입력과 출력 (화면 capture 시킬 것)
 2. 결과 분석
4. 소스코드

□ 과제 제출

■ 파일 이름 작명 방법

- DS1_10_학번_이름.zip

- 폴더의 구성

- ◆ DS1_10_학번_이름

- 프로그램

- 프로젝트 폴더 / 소스

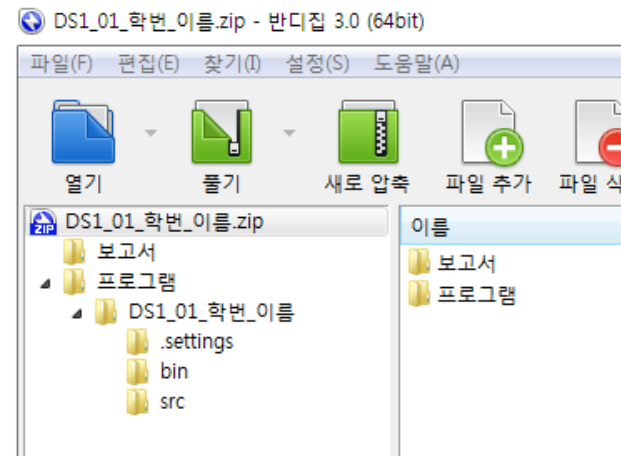
- 메인 클래스 이름 : DS1_10_학번_이름.java

- 보고서

- 이곳에 보고서 문서 파일을 저장한다.

- 입력과 실행 결과는 화면 image로 문서에 포함시킨다.

- 문서는 pdf 파일로 만들어 제출한다.



[제 10 주] 끝



